Lab 3 – Configure Network Segregation Policies

SPR500-NBB-Group 5

Ayaan Hirsi (17317721)

Table of Contents

Table of Figures

Executive Summary

This lab focused on automating firewall deployment and DNS configuration across all machines using Ansible. The goal was to make network management more efficient, consistent, and secure.

Ansible was used to automatically deploy nftables firewall configuration to every host in the internal network, including the Admin, User, Active Directory/DNS machine, and Container Host machines. Each machine received firewall rules suited to its specific role, while Ansible ensured that all configurations were applied correctly and consistently. This approach reduces manual errors and improves control over the system's security posture.

In addition to the firewall automation, a split DNS configuration was implemented. The previous public DNS service was removed, and the single internal DNS server was configured to handle both internal and external queries. With this setup, when an internal host performs a lookup, it receives the internal IP address (Container Host). However, when an external user performs an nslookup for a public site such as *corpweb.g5.external,* it correctly resolves to the external router's IP address. This behavior was achieved even though there is only one DNS machine in the network, showing that the split DNS configuration works effectively without needing separate servers.

Overall, this lab resulted in a secure and automated network environment. Firewall management is now handled through centralized automation, and the DNS configuration provides proper name resolution for both internal and external users while maintaining network segmentation.

Infrastructure Overview

The lab environment was built as a segmented internal network managed through Ansible. The Admin VM serves as the Ansible control node, responsible for managing and deploying firewall rules to all other machines within the internal network. Using Ansible automation ensured consistency across hosts, simplified rule enforcement, and prevented configuration drift.

The Active Directory/DNS machine acted as both the domain controller and central DNS resolver. The DNS configuration was redesigned to support a split DNS setup. Two logical zones were created: one for the internal network and another for external resolution. This configuration allows internal hosts to resolve local names within the corporate domain, while external users performing lookups for domains ending in .external (for example, *corpweb.g5.external*) receives the public IP address of the external router. This behaviour occurs even though there is only a single DNS machine, as the split-DNS logic enables it to respond differently depending on the query source.

All machines, including the Admin User, Active Directory, and the container Host systems, were configured with nftables firewalls were deployed and updated automatically via Ansible. The Container Host operates as the reverse proxy, hosting internal and public web services through the HTTP protocol.

The network is designed so that the core router connects the internal segments to the external router for internet access, while the internal DNS firewalls ensure proper name resolution and traffic control within each subnet.

Key Points

- The Admin VM functioned as the Ansible manager, applying firewall configurations across all internal hosts.
- Nftables was used for all firewall rules, ensuring a stateful, consistent, and secure environment.
- The AD/DNS machine manages both internal and external zones using a split-DNS setup.
- External lookups such as *corpweb.g5.external* resolves to the external router's IP address, while internal queries return the Container Host's IP address.
- The environment uses only one DNS machine, logically divides to server both internal and external clients
- The Container Host provides internal and public web services.
- All configurations were deployed, validated, and maintained through Ansible automation to ensure synchronization and avoid manual errors.

Evidence of Successful Deployment (Configurations)

Split-DNS Configuration

- Internal Zones

Figure 1 shows the internal DNS zone configuration within the Active Directory Domain Services. The DNS server *(l1ad.g5.local)* manages the internal domain namespace, providing name resolution for all internal hosts and services in the G5 network. In this configuration, each machine and service within the internal environment is registered with its hostname and corresponding IP address. For example, entries such as *corpweb, fileshare, intuser, and intweb represent,* represent different internal resources, while records like *admin, conthost, and dc1* correspond to the administrative, container host and domain controller machines, respectively. The internal DNS zone allows all hosts within the internal network to communicate efficiently using host names rather than IP addresses. This configuration ensures proper internal name resolution and forms the basis for the split-DNS setup implemented in the lab.

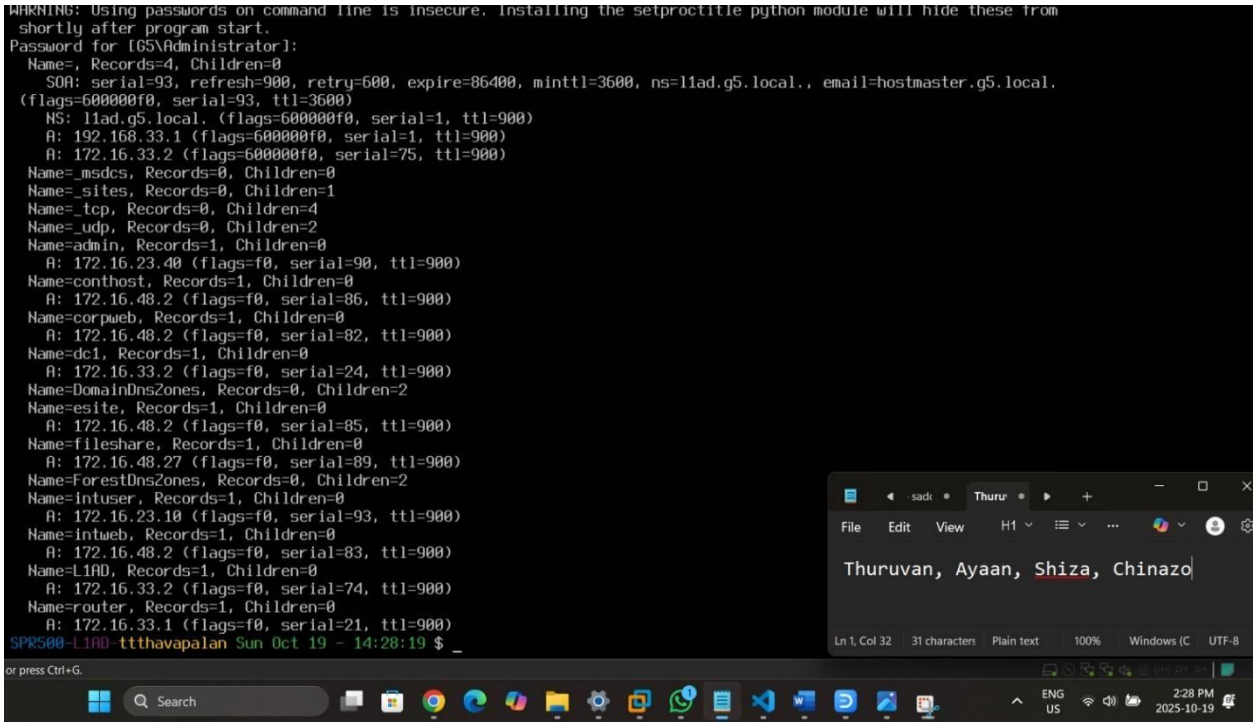

Figure 1: Internal DNS Zone records.

- External Zone

Figure 2 shows the external DNS zone configuration that complements the internal DNS setup in the split-DNS design. The DNS server (*liad.g5.local*) is also the authoritative for the external domain (g5.external). In this configuration, external records such as *corpweb, esite, fileshare, and intweb* all map to the same public-facing IP address, 172.16.2.33, *172.16.2.33* which represents the external router interface. This setup ensures that when an external user performs a DNS lookup (for example, corpweb.g5.external), the DNS server responds with the external IP rather than an internal one. This design allows external access to public-facing web services while keeping internal IP addresses hidden and protected. By using a single DNS server to manage both internal and external zones, the configuration achieves logical separation of namespaces without requiring additional DNS infrastructure.

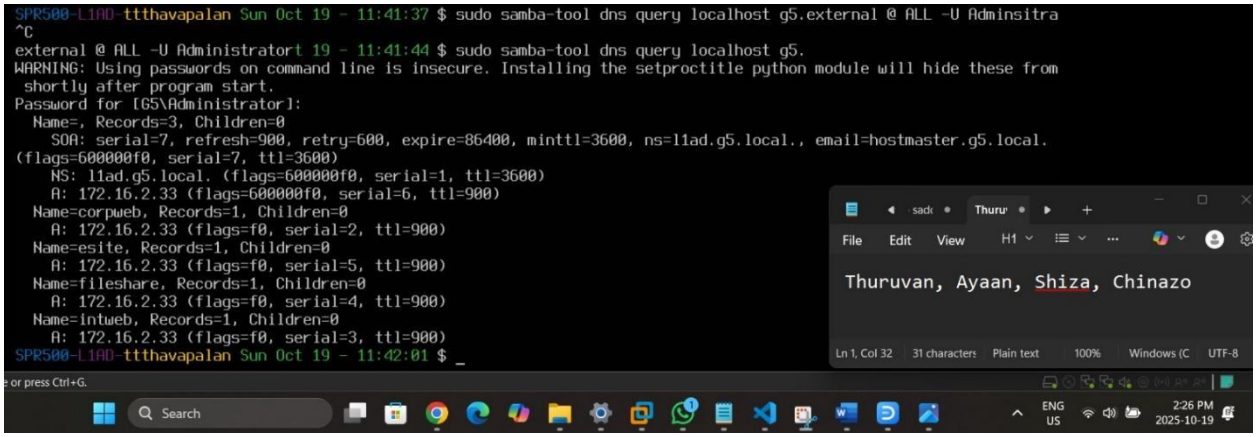

Figure 2: External DNS Zone Configuration

Ansible Configuration

- Inventory File

Figure 3 shows the Ansible inventory file (*inventory.ini*) used to define and categorize all managed hosts within the lab environment. Each host entry specifies its hostname (*FQDN*), Ansible user, and the group it belongs to, such as *[core_router], [ad], [container], [user], and [admin].* The structure allows Ansible to apply host-specific configuration and firewall rules dynamically during playbook execution. The admin host served as the Ansible control node, which securely connected to all other machines using SSH authentication and deployed firewall rules to each of them.

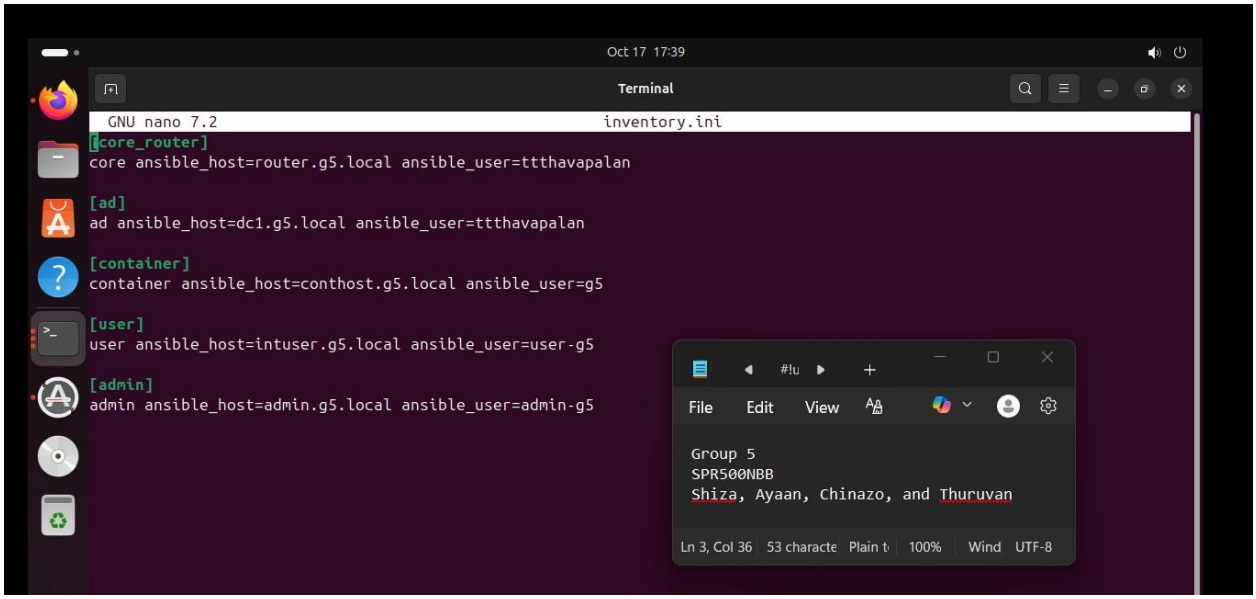

Figure 3: Ansible Inventory File Configuration

- Playbook File

Figure 4 shows the initial section of the Ansible Playbook used to automate the deployment and configuration of stateful firewalls across all network hosts. In this lab, the admin VM acted as the Ansible control node and applied the rules to all managed hosts in the internal network. This part of the playbook ensures that the nftables package is installed on each target system, verifies that the nftables service is enabled and running, and then deploys the correct firewall configuration file *({{ inventory_hostname }}_nft.conf)* from the files directory to /etc/nftables.conf path on each host. The variable substitution (*{{ inventory_hostname }}*) allows Ansible to dynamically select the configuration for each host, ensuring host-specific firewall management.

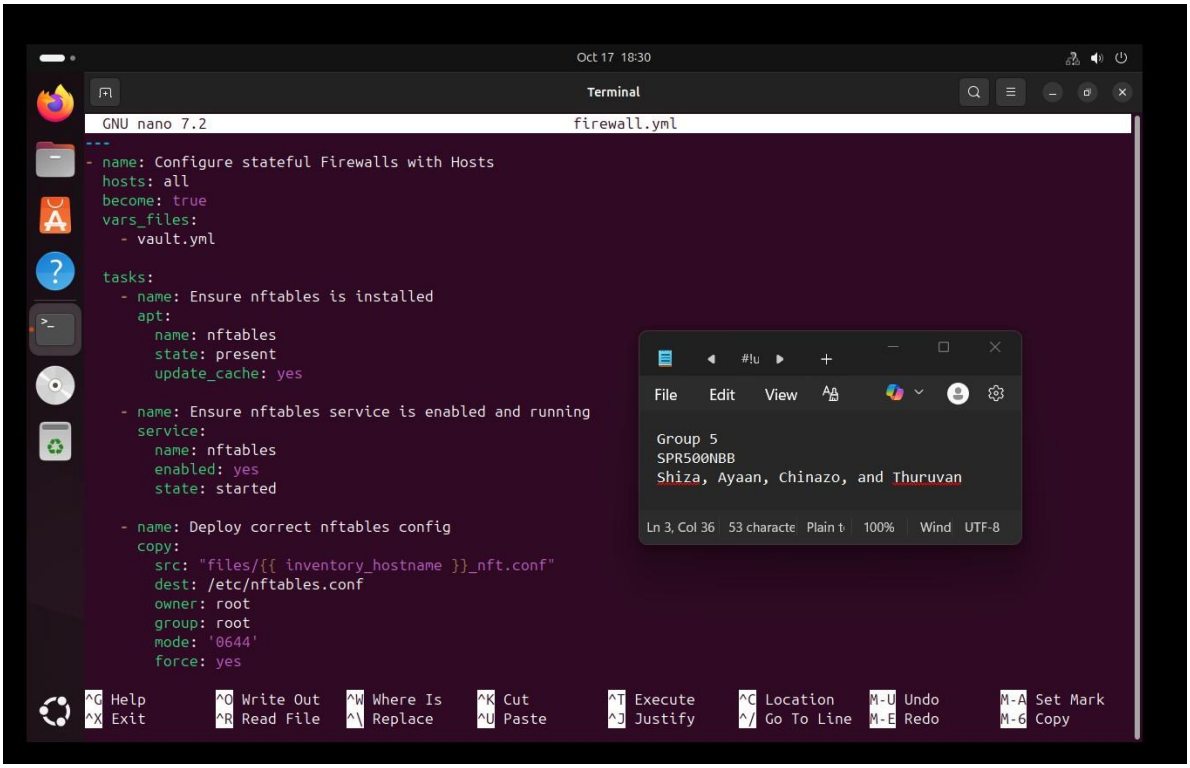

Figure 4: Ansible Configuration for Firewall Automation

Figure 5 shows the second part of the Ansible Playbook that applies, verifies, and restarts the nftables firewall configuration on all managed hosts. After copying each hosts' configuration file, the playbook flushes existing firewall rules to ensure a clean state on all machines except the container host, since it

runs Docker and relies on Docker-managed NAT and bridge rules for container networking. On the container host, the playbook skips the flush step to avoid disrupting Docker's internal networking while still safely applying the custom nftables rulese. If any configuration file changes are detected, the nftables service is automatically restarted to apply the latest firewall policies. Finally, a debug task prints a summary showing whether each host's firewall configuration was updated, confirms that the nftables service is actuve and enabled, and displays the configuration file path of verification.

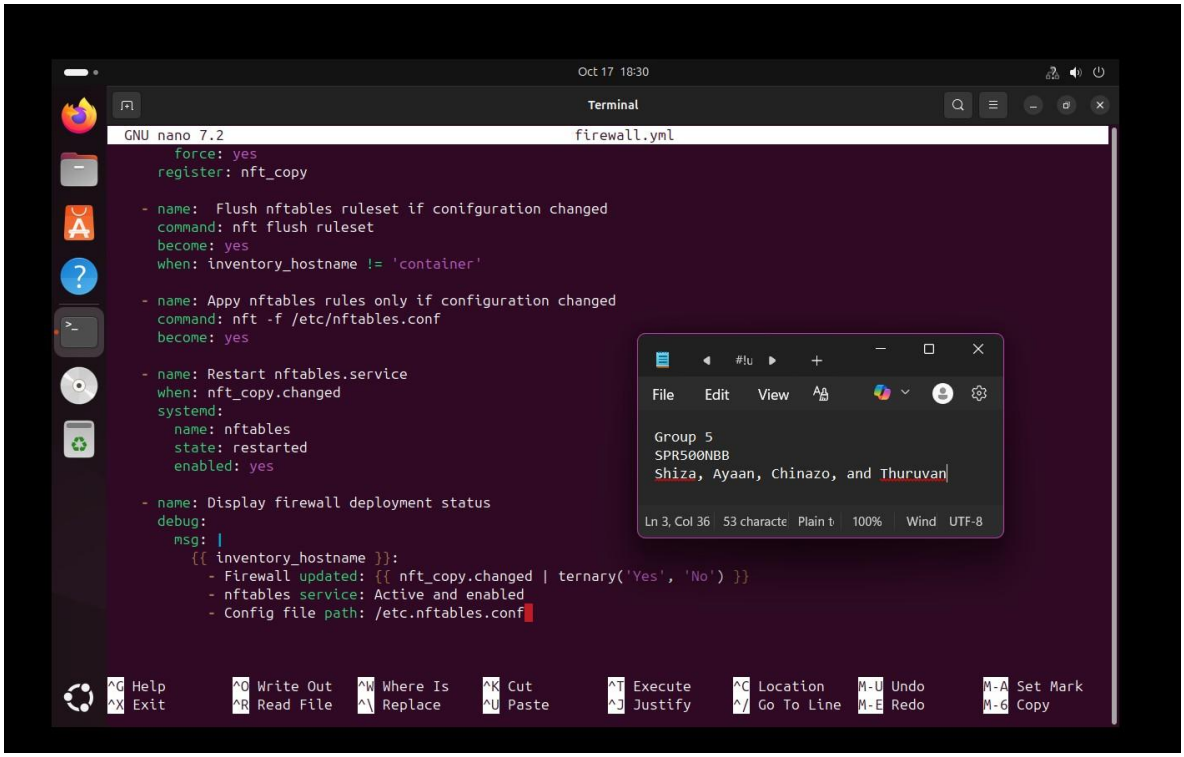

Figure 5: Ansible Playbook – Firewall Application and Validation

Firewall Configuration Files

To organize and deploy these configurations efficiently, a dedicated directory named *files/* was created within the Ansible project structure. Each host (*admin, AD, User, Container, and Core Router*) has its own firewall configuration file stored in this directory (e.g., *user_nft.conf, admin_nft.conf, etc.*). During the playbook execution, Ansible automatically copies the appropriate configuration file from the *files/* directory to */etc/nftables.conf* on the corresponding host, applies the rules, and then restarts the nftables service as required. This directory-based approach ensures consistency, easy management, and clear separation of firewall policies for each host.

- User Machine

Figure 6 shows the nftables firewall configuration file for the User virtual machine after the core router was designated as the primary point of traffic control and network segmentation. Since the router now enforces most filtering policies, the user firewall was reduced to essential stateful rules that maintain local protection while allowing necessary connectivity. The configuration accepts local loopback traffic, tracks established and related sessions, and then drops invalid packets to maintain stability. It permits SSH access (port 22) for secure management and allows outbound HTTP, HTTPS, SMB, and DNS traffic to ensure that the user can reach internal and external services as needed. This minimal yet effective configuration prevents unnecessary overlap with router-based rules while ensuring that the User machine remains protected and functional within the network.

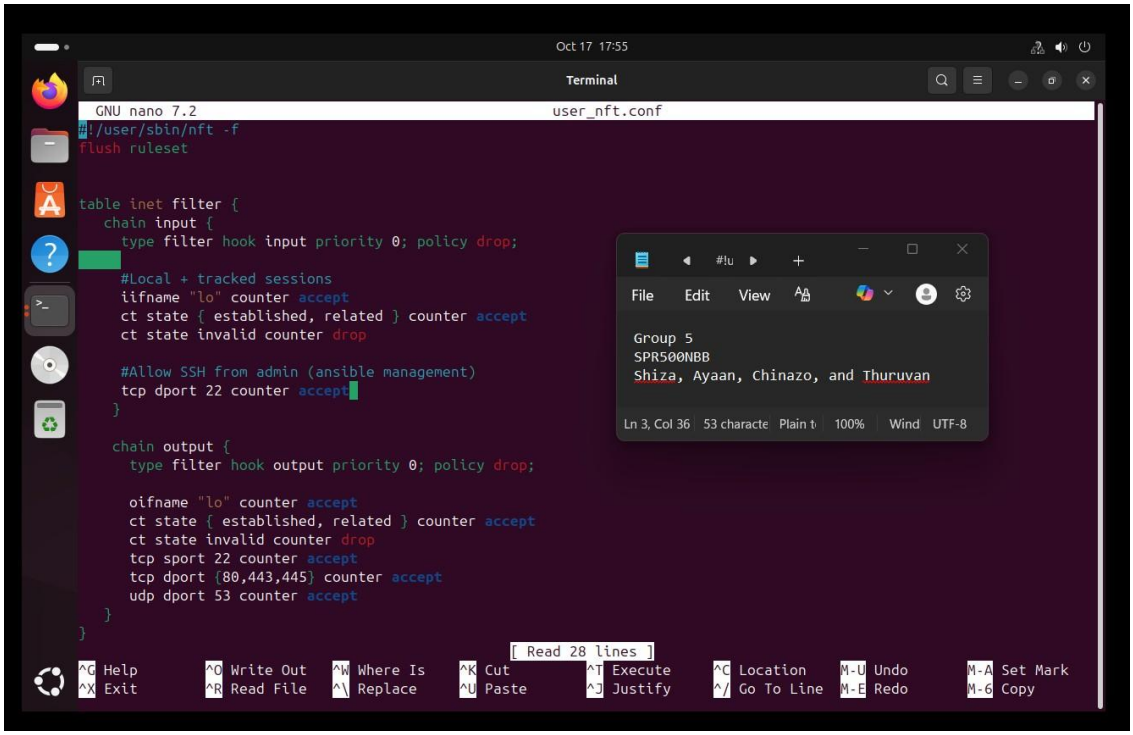

Figure 6: User Machine nftables Configuration

- Admin Machine

Figure 7 displays the nftables firewall rules deployed on the Admin VM through Ansible. The configuration enforced a default-deny policy for both input and output chains while allowing loopback traffic, established or related connections, and SSH (port 22) for remote management. It also permits essential outbound services such as HTTP/HTTPS (ports 80, 443), SMB (445), and DNS (53). These rules secure the administrative node while ensuring it can manage internal hosts and reach required update repositories efficiently.

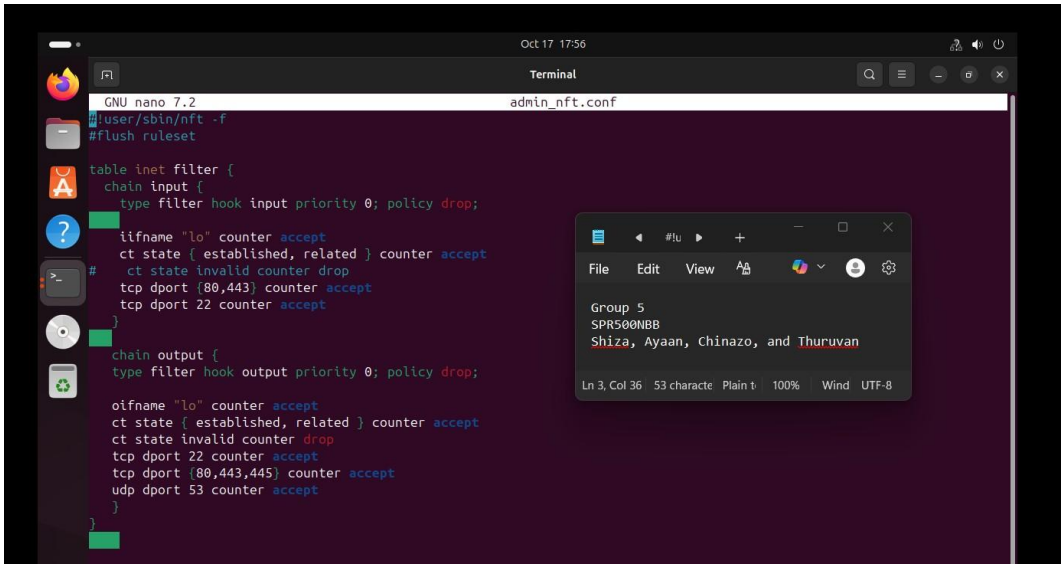

Figure 7: Admin Machine nftables Configuration

- AD Samba Machine

Figure 8 shows the firewall rules configured on the Active Directory and DNS server. The rules enforce a default drop policy again, while allowing important inbound and outbound communication for SSH (22), DNS (53), and directory services. The ports listed enable Active Directory replication, authentication, (Kerberos, LDAP, SMB) and DNS operations. Outbound traffic for DNS and web access (HTTP/HTTPS) is also permitted, ensuring domain resolution and updates function correctly while also maintaining a secure, stateful filtering policy.

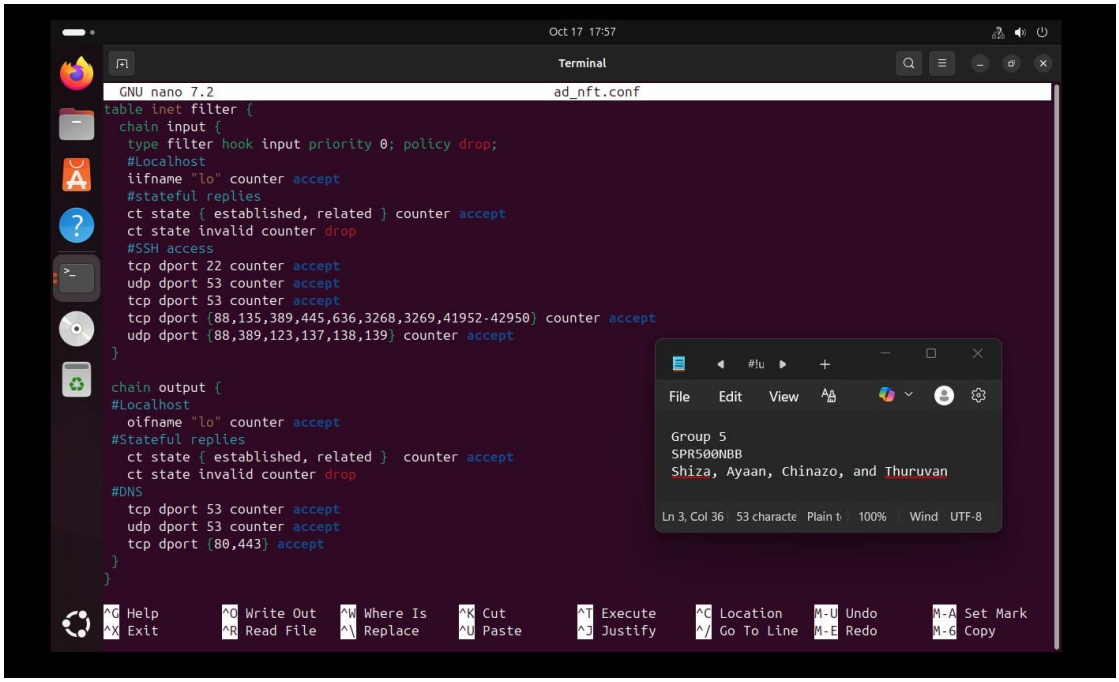

Figure 8: AD Samba Machine nftables Configuration

- Container Machine

Figure 9 shows the nftables rules applies on the Container Host, which serves as the reverse proxy hosting both internal and public web services. The configuration enforced a default drop policy, only allowing necessary inbound and outbound connections. SSH (22) is enabled for secure management, while HTTP/HTTPS (80, 443) traffic is allowed to support web access. DNS (53) queries are permitted for name resolution and established or related sessions are accepted to maintain connection statefulness. This setup ensures that the container host remains accessible for web and administrative functions while minimizing exposure to unnecessary network traffic.

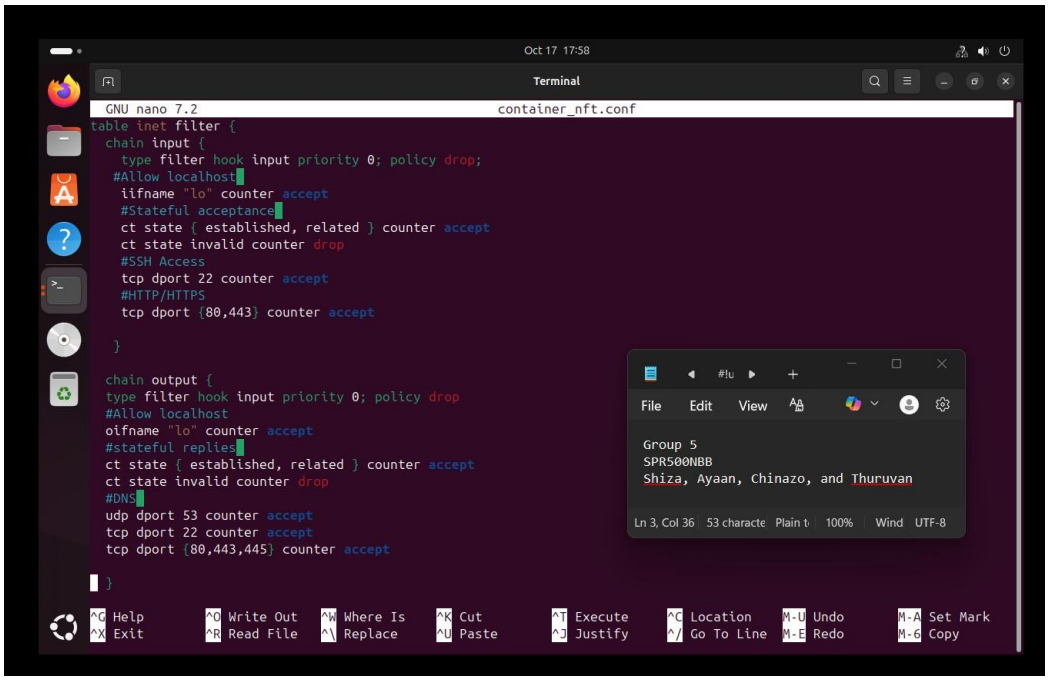

Figure 9: Container Host Machine nftables Configuration

- Core Router Machine

Figure 10 shows the configuration that defines multiple IP sets for all internal external subnets, allowing structured control access network segments. The input chain applies a default drop policy to block unauthorized access while allowing loopback, established, and related traffic. It specifically permits SSH connections from internal administrators, OSPF for routing updates, and DHCP relay to support dynamic IP assignments. These rules secure the router's management plan while ensuring critical routing and addressing services function correctly.

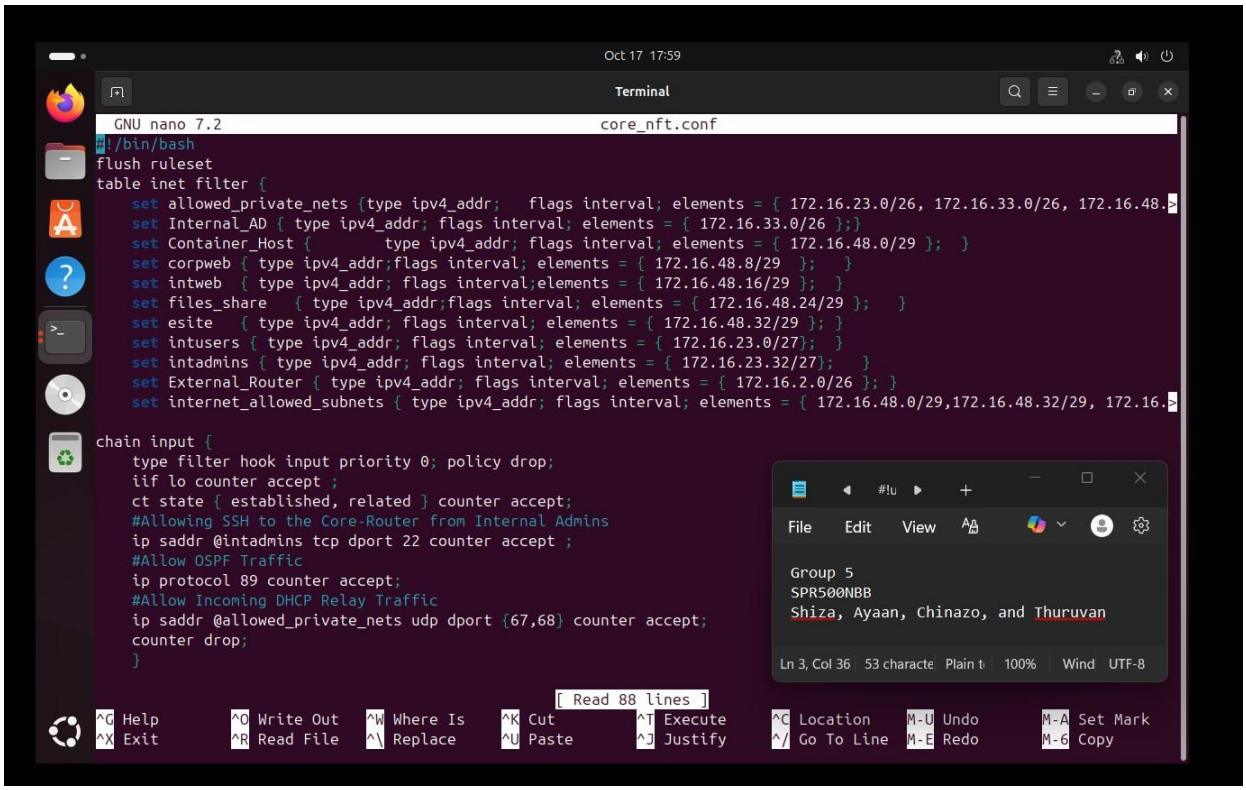

Figure 10: Core Router Machine nftables Configuration (part 1)

Figure 11 shows the output chain which enforces strict outbound control from the router, maintaining a policy drop stance for nonessential traffic. It allows only trusted services, including OSPF, DHCP relay replies, and SSH access to internal admin hosts. Loopback and established connections are accepted to maintain system stability. This setup ensures the router communicates only with authorized peers, which prevents unnecessary external exposure.

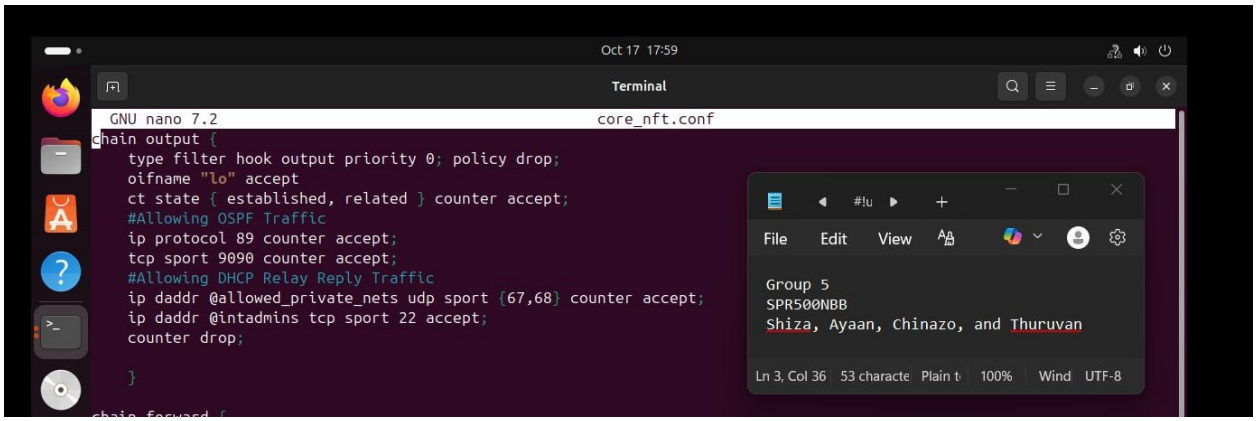

Figure 11: Core Router Machine nftables Configuration (part 2)

Figure 12 shows the first section of the forward chain which governs traffic routing between VLANs and zones. It permits DNS queries from internal networks to the AD server, allows administrative

Figure
and directory replication traffic between AD and internal admins, authorizes user access to Samba and authentication services. The configuration also includes rate-limiting rules for SYN packets to mitigate potential flooding attacks and permits HTTP/HTTPS communication from internal subnets to the container host. These rules also enforce segmentation while supporting essential business operations.

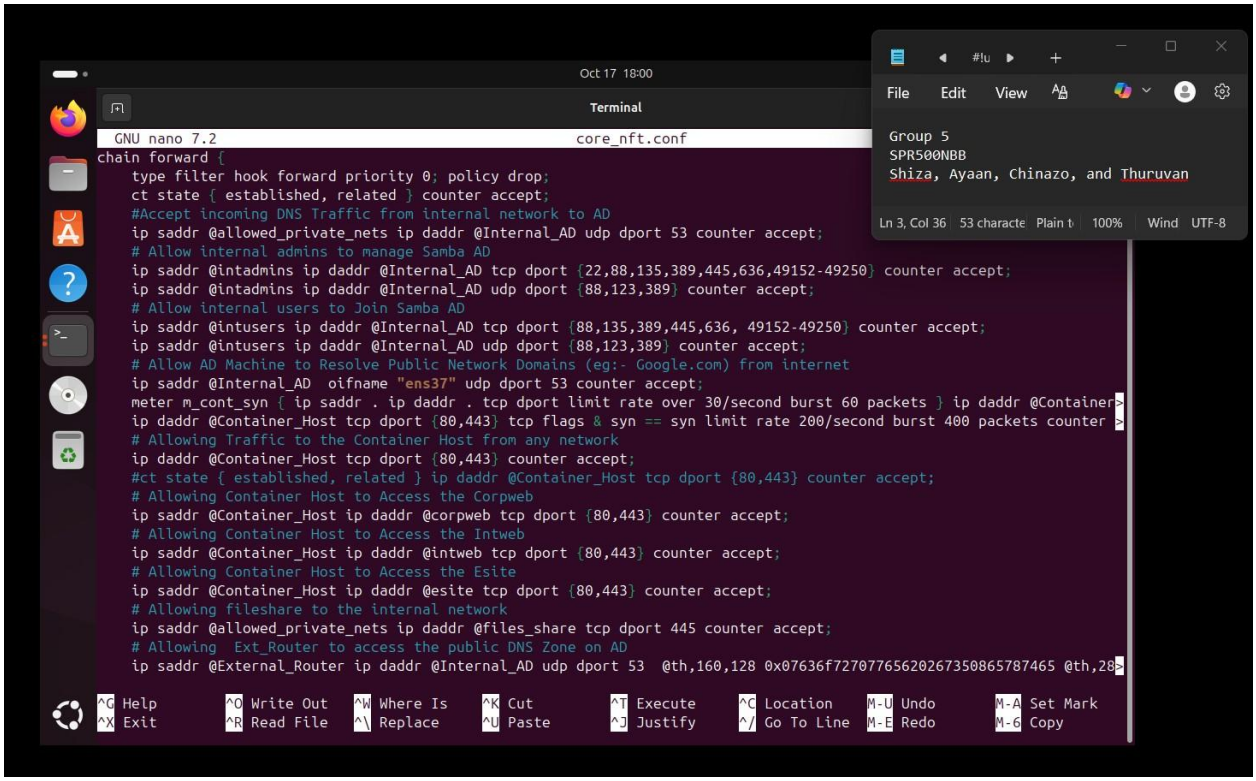

Figure 12: Core Router Machine nftables Configuration (part 3)

13 shows the final portion of the forward chain manages inter-zone access and security logging. It allows the container host to reach web services such as corpweb, intweb, and esite, while restricting fileshare access to internal traffic only. The external router can query the internal DNS to maintain public name resolutions. Outbound web and DNS traffic from external networks is permitted,

Figure
but SSH from regular users to the container host is explicitly blocked. A logging rule at the end records all dropped packets, ensuring visibility and accountability in traffic filtering.

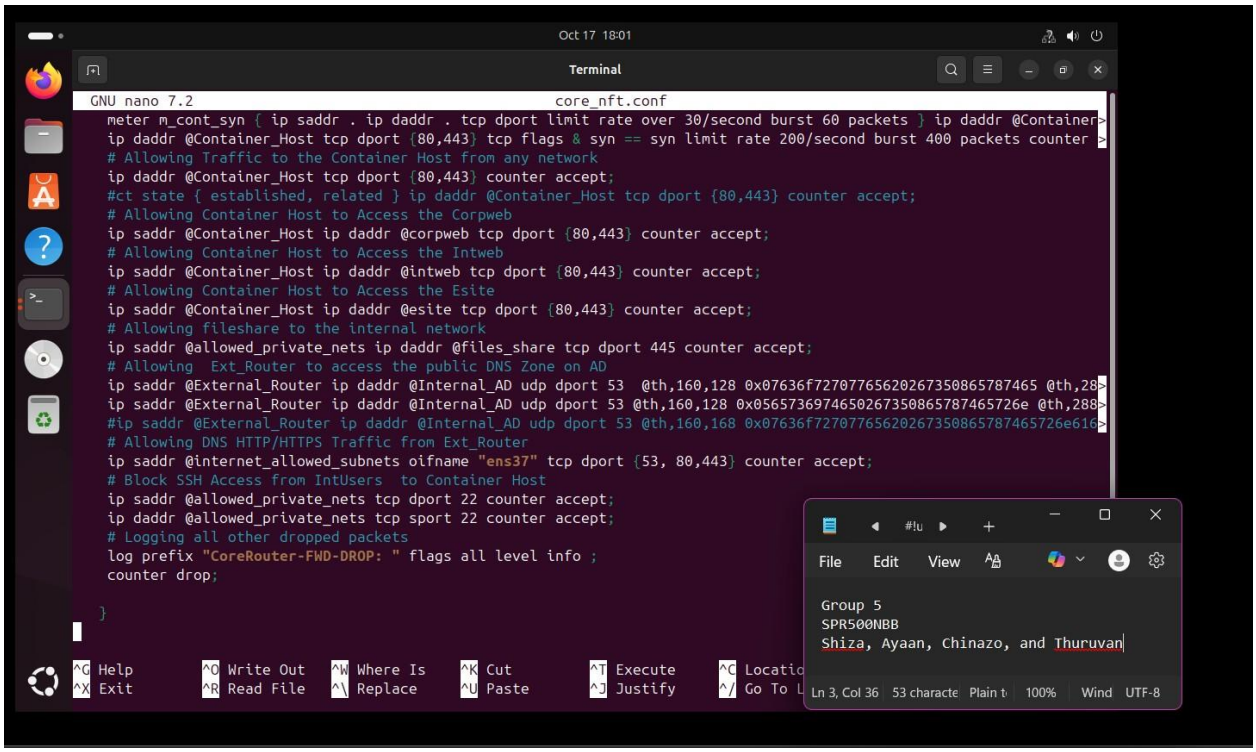

Figure 13: Core Router Machine nftables Configuration (part 4)

External Router Firewall Configurations

Figure 14 is the NAT table with two chains, prerouting and postrouting. Prerouting handles incoming packets and post routing handles outgoing packets.

- The DNAT rules rediret udp/tcp DNS from the external router to the public DNS, it also redirects tcp HTTP traffic from the external router to the container host.
- The SNAT rules masquerade traffic from 172.16.2.0/27 and 172.16.2.32/27 so the external/internal devices can reach the internet.

Figure 14: NAT TABLE: External Router nftables Configuration Figure

15 is the filter table which has the input/output chain.

- The input chain accepts traffic on the loopback interface
- Allows packets using protocol 89 (OSPF)
- Permits established and related connections (for replies) - Allows ICMP (for ping and network diagnostics)

Figure 15: FILTER TABLE: External Router nftables Configuration

Figure 16 is the forward chain which controls traffic passing through the router.

- It allows established and related connections making it stateful
- HTTP traffic allowed from 172.16.2.0/27 and 172.16.2.32/27 to 172.16.48.2
- DNS traffic allowed from both subnets (internal/external)
- Allows internet access HTTP/HTTPS for both subnets
- Allows ICMP from both subnets
- Keeps the network secure by dropping all other forwarded traffic.

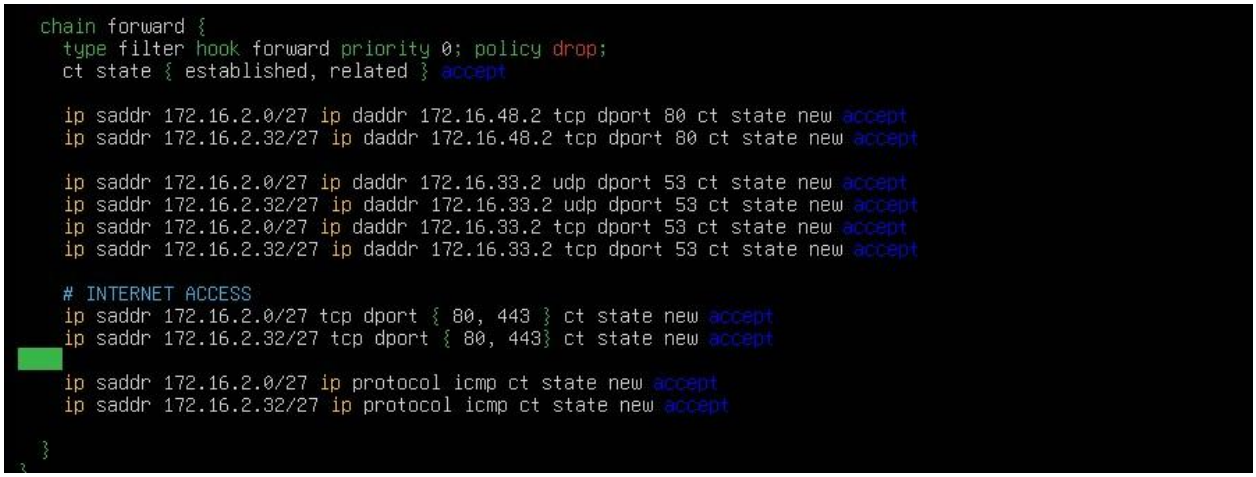

Figure 16: FILTER TABLE: External Router nftables Configuration

Test Cases

Figure 17 shows the initial phase of the Ansible playbook execution, where connections are successfully established to all managed hosts: *admin*, *ad*, *core*, *container*, and *user*. The playbook confirms that nftables is installed and the service is both active and enabled across all systems. This verifies that Ansible communication and privilege escalation are functioning properly. Each host reports "ok," indicating no connectivity or configuration errors during setup.

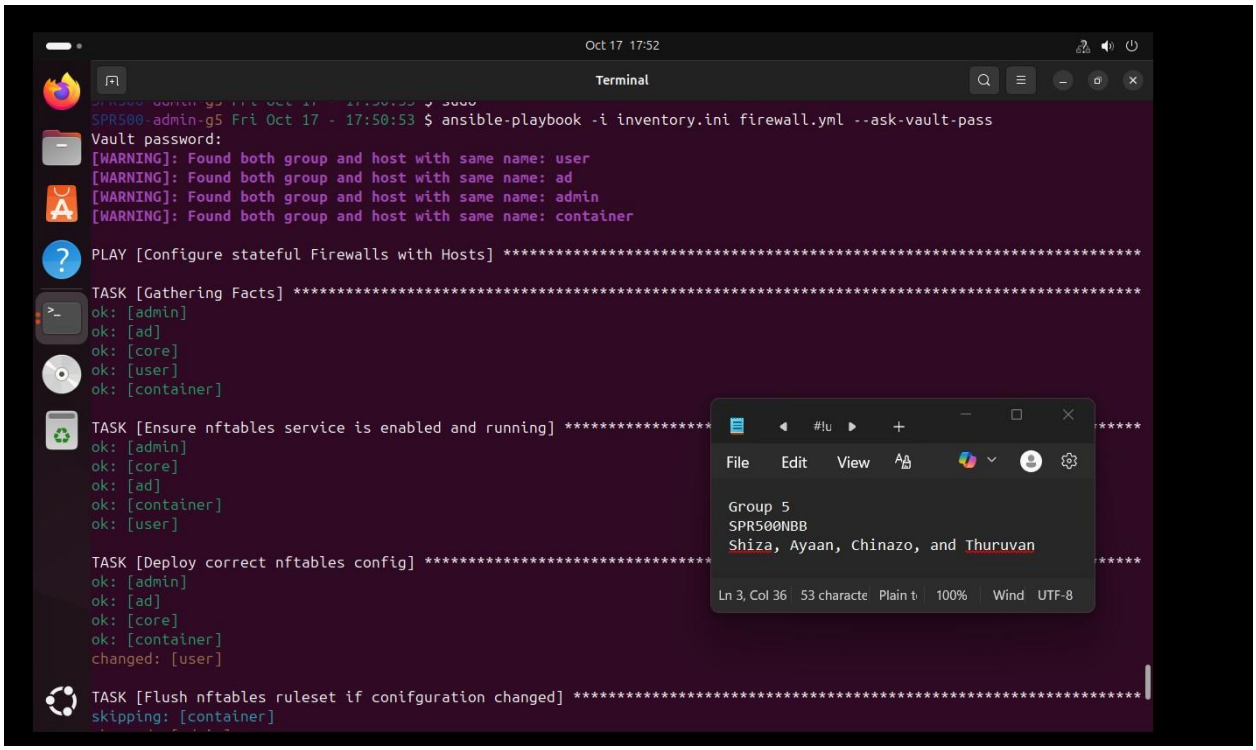

Figure 17: Ansible Playbook Execution – Initial Configuration and Service Verification

Figure 18 illustrates the deployment and application fo the nftables configuration files to all managed hosts. The task outputs show the playbook copied the host-specific firewall files into */etc/nftables.conf,* with changes detected on the user host. The flush task was skipped for the container host to preserve existing Docker networking rules, while all other machines safely reloaded their rulesets. The conditional logic successfully ensured that sensitive systems like the container hosts were not reset during deployment.

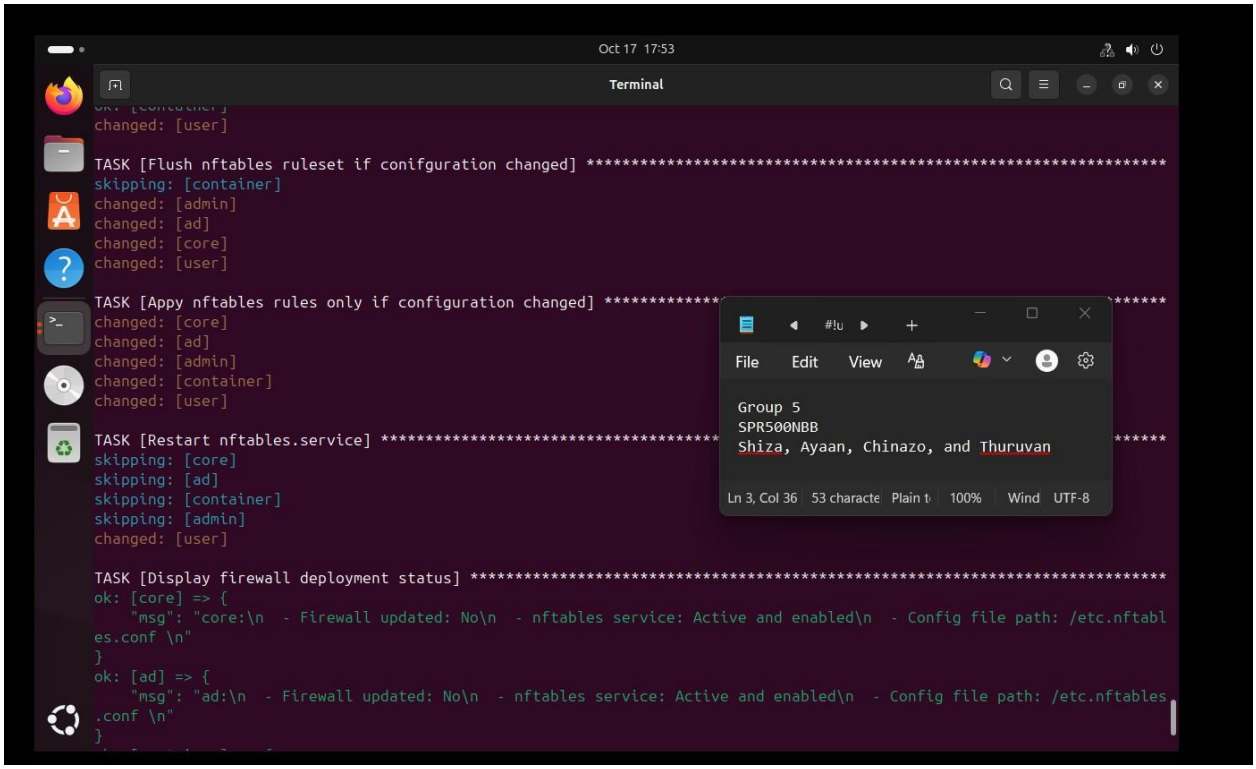

Figure 18: Firewall Configuration and Conditional Rule Application

19 displays the final verification step, where Ansible confirms that the nftables service is active and enabled on all hosts. The debug output summarizes each host's configuration status, showing that the *user* machine's firewall was updated, while others remained unchanged, meaning their

Figure
configurations were already up to date. The play recap at the bottom shows unreachable=0 and failed=0, confirming a fully successful deployment across the entire environment.

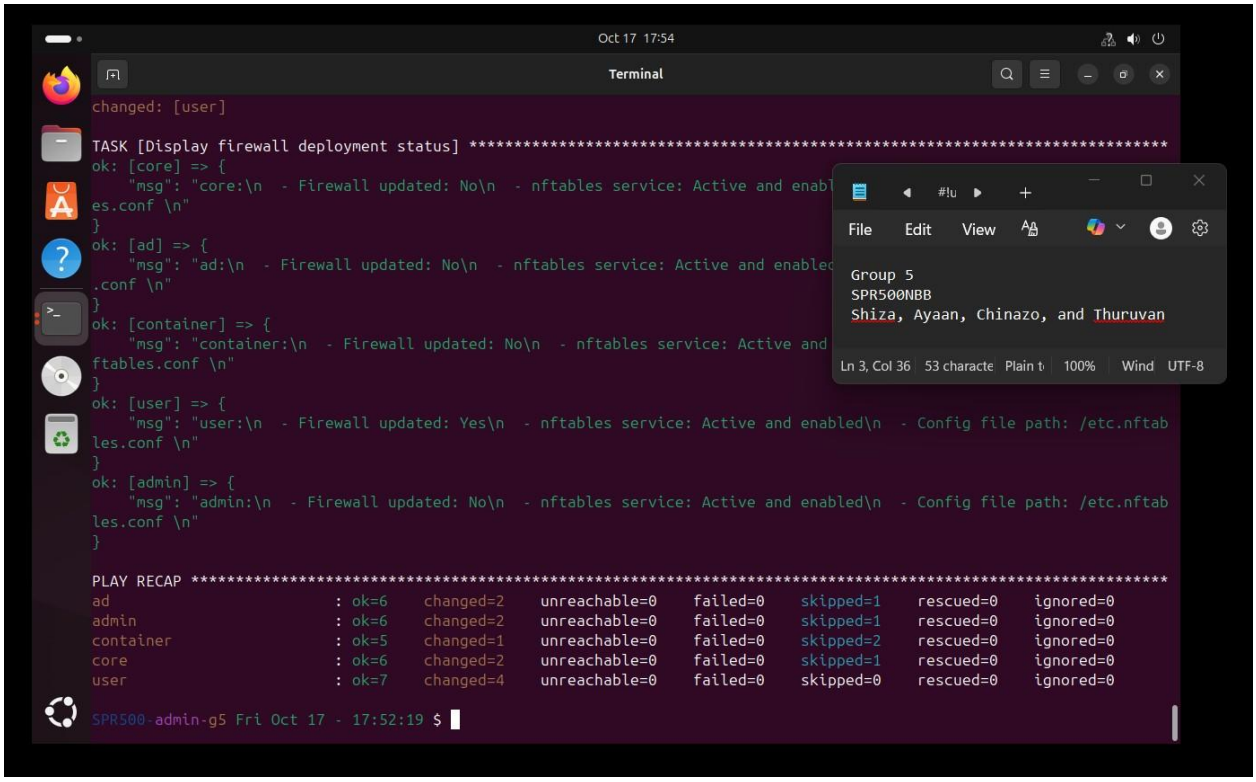

Figure 19: Firewall Status Verification and Play Recap Summary

20 shows a successfully functionality test conducted from the admin machines after running the playbook and the firewalls were applied across the network. The admin user was able to access

Figure
multiple internal web services, including *CorpWeb, E-Site, and IntWeb,* confirming that the firewall rules were properly configured to allow legitimate traffic within the internal network. This test validates that critical internal communication, DNS resolution, and service access were not disrupted by the new stateful firewall policies. It demonstrates that the Ansible-based deployment maintained required connectivity for administrative management and internal business applications, proving the configuration's correctness and reliability.

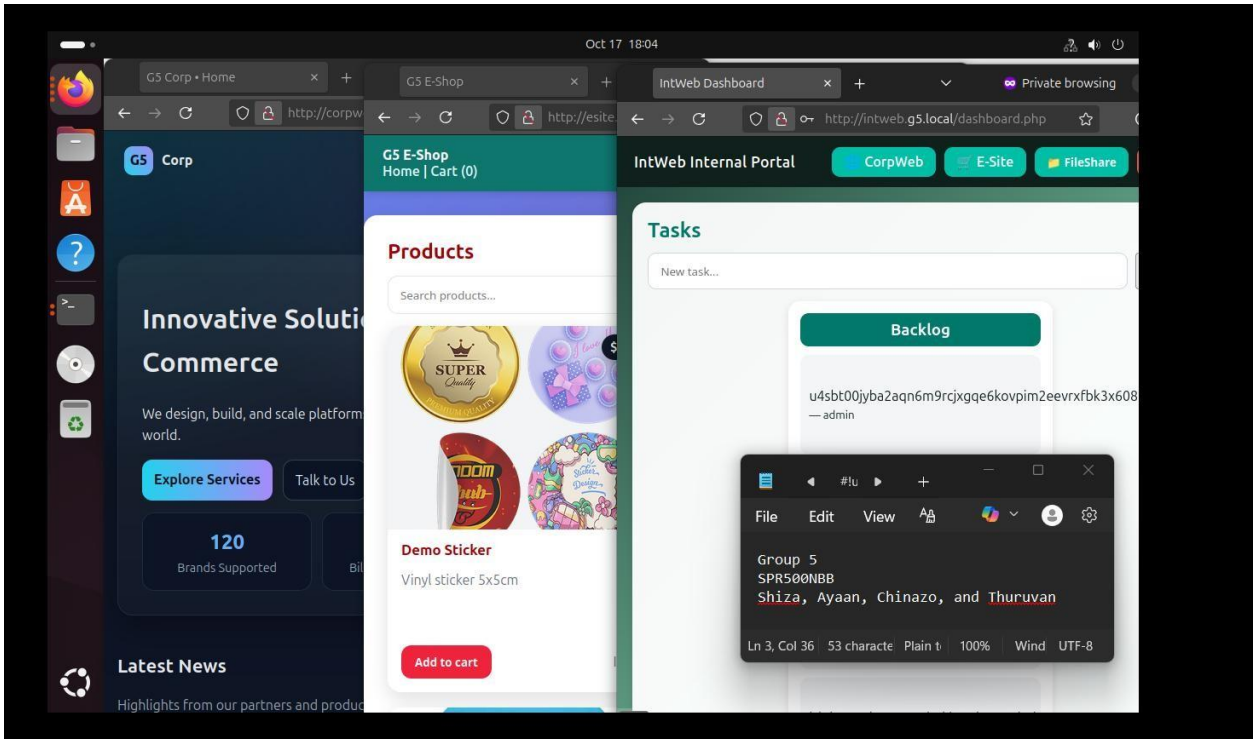

Figure 20: Web Application Accessibility Test after Firewall Deployment

Figure 21 shows a test which demonstrates that the firewall's SYN flood protection rules worked effectively. The hping3 command (sudo hping3 -S -p 80 -i u10000 -c 300 172.16.48.2) was used to simulate a SYN flood attack by sending 300 rapid TCP SYN packets to port 80 of the container host. The firewall rule tcp flags syn ct state new limit rate 200/second burst 400 accept; limits the number of new SYN packets the host can accept per second, dropping any excess packets with the rule tcp flags syn ct state new counter drop. As shown in the test results, although 300 packets were transmitted, only 194 were received, indicating that the remaining packets were dropped by the nftables firewall as part of its rate-limiting defense. This confirms that the implemented firewall rule successfully mitigated the simulated flood by allowing normal traffic while blocking excessive connection attempts.

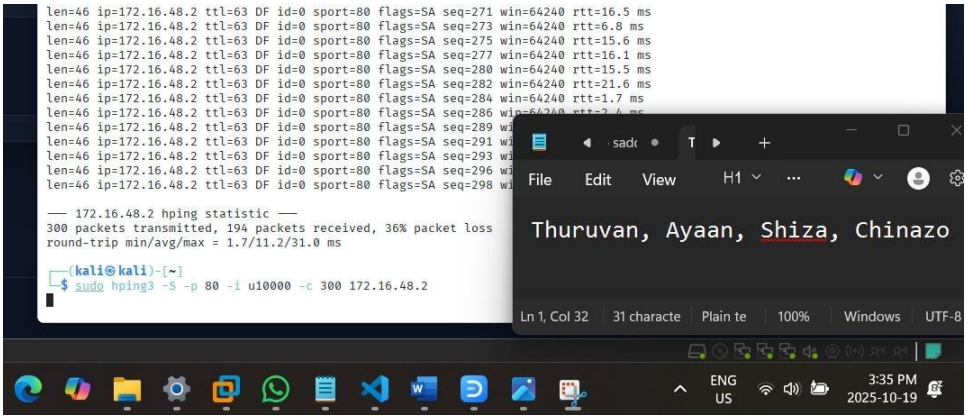

Figure 21: SYN flood attack test showing the nftables firewall successfully limiting excessive SYN packets through rate-limiting rules.

Figure 22 shows the results of DNS resolution tests performed from an external machine to validate the split-DNS configuration. The external DNS server (*172.16.33.2)* successfully resolves public-facing domains such as *corpweb.g5.external* and *esite.g5.external* to the correct IP address *(public IP of the external router),* demonstrating that these services are accessible from outside the internal network. In contrast, queries for internal-only records like *intweb.g5.external* and *fileshare.g5.external* fail to respond, confirming that the DNS setup correctly restricts internal hostnames from external access and effectively enforces network segmentation.

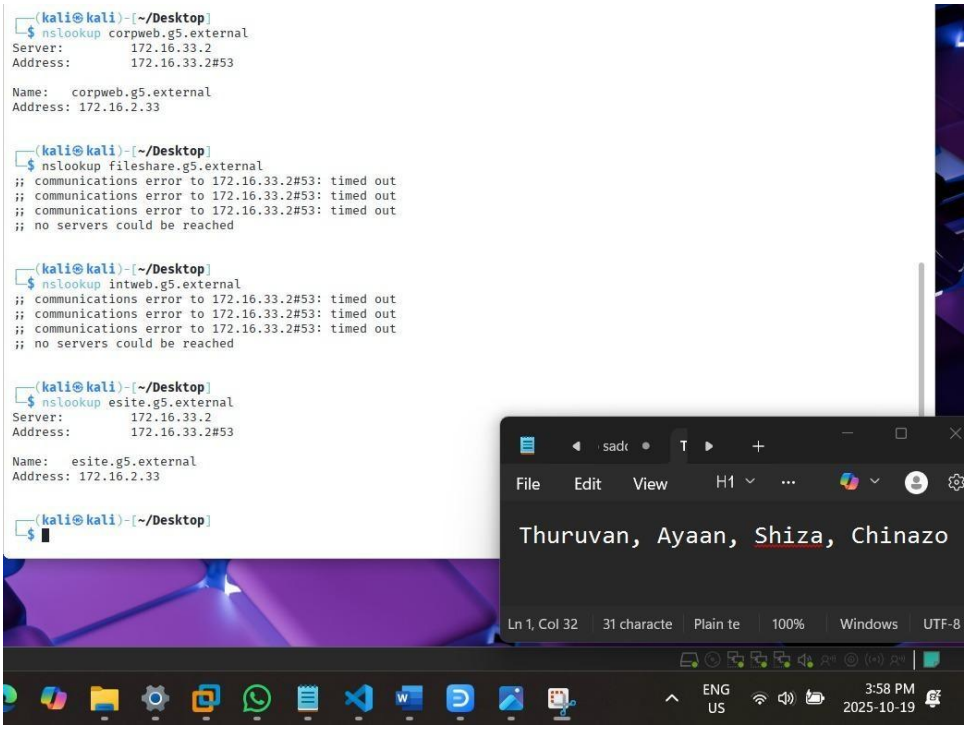

Figure 22: Split DNS verification from an external client.

Figure 23 shows the verification of nftables rules on the Core Router, confirming that DNS traffic from the External Router to the Internal Active Directory (AD) server is being accepted as intended. The command *sudo nft list ruleset | grep "@th"* filters the output to show matching rules, indicating successful packet counts and byte transfers on UDP port 53. This confirms that the configured firewall rule is correctly allowing DNS queries between the two routers, ensuring proper internal name resolution while maintaining controlled access.

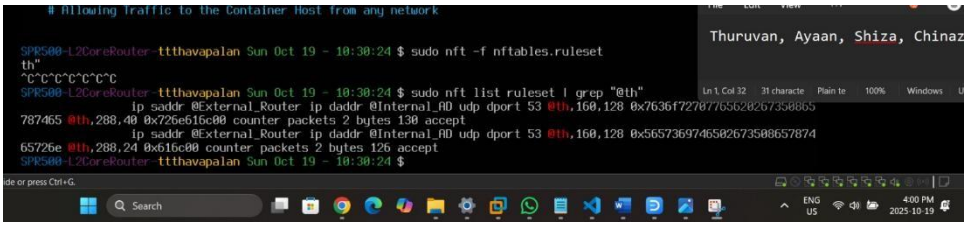

Figure 23: Verifying DNS traffic allowance between routers.

Conclusion

In conclusion, this lab successfully demonstrated the implementation of a secure, automated, and segmented network environment using Ansible and nftables. Each host, including the Admin, User, AD/DNS, Container Host, and Core Router, was configured with tailored firewall rules that aligned with its role in the infrustructure. The core router effectively enforced inter-zone boundaries through VLAN segmentation, NAT, and stateful filtering, while the split-DNS configuration ensured proper internal and external name resolution without any other servers. Automation through Ansible minimized configuration errors, maintained consistency across devices, and simplified future updates. Overall, the deployment achieved the lab's objectives by improving security, scalability, and manageability, creating a resilient architecture that reflects real-world enterprise network practices.

References

Byström, J., & Lundberg, L. (2021). Security considerations in container-based infrastructures. *Journal of*

Cloud Computing, 10(1), 1–15. https://doi.org/10.1186/s13677-021-00257-8

Docker. (n.d.). Docker compose. Docker Documentation. Retrieved September 15, 2025, from

https://docs.docker.com/reference/cli/docker/compose/

Goldlust, S. (2022, February 17). ISC DHCP 4.1 manual pages - dhcpd.conf. Internet Systems

Consortium (ISC). Retrieved September 15, 2025, from https://kb.isc.org/docs/isc-dhcp-41-manualpagesDhcpdconf

KL, A. (2023, November 1). *Step-by-step procedure to set up an Active Directory on Ubuntu*. The Sec

Master. Retrieved September 15, 2025, from https://thesecmaster.com/blog/step-by-step-procedure-tosetup-an-active-directory-on-ubuntu

"Macvlan network driver." (2025, February 5). Docker Documentation.

https://docs.docker.com/engine/network/drivers/macvlan/

Red Hat. (2023). *Setting up and configuring a BIND DNS server*. In *Red Hat Enterprise Linux 9: Managing networking infrastructure services*. Retrieved September 15, 2025, from

https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/managing_networking_infrastructure_services/assembly_setting-up-and-configuring-a-bind-dns-server_networkinginfrastructureservices

Tran, T. (2022, September 15). *How to configure Nginx as a reverse proxy on Ubuntu 22.04*.

DigitalOcean Community Tutorials. Retrieved September 15, 2025, from

https://www.digitalocean.com/community/tutorials/how-to-configure-nginx-as-a-reverse-proxyonubuntu-22.04

Patel, R., & Gupta, A. (2020). Improving enterprise security with Active Directory and DNS hardening.

International Journal of Computer Applications, 176(38), 12–18. https://doi.org/10.5120/ijca2020920011